# Reducing Hallucinations in Retrieval-Augmented Question Answering Systems


## Abstract:

Large Language Models (LLMs) often generate incorrect or unsupported information, known as hallucinations. This project implements a Retrieval-Augmented Generation (RAG) based Question Answering system to reduce hallucinations by grounding responses in a fixed document knowledge base. The system compares basic question answering with retrieval-augmented answering and evaluates hallucination detection and confidence scoring mechanisms.


## 1. Introduction

AI-based question answering systems are widely used in education, search engines, and chatbots, but traditional large language models can sometimes produce incorrect or unsupported responses, a problem known as hallucination. To address this limitation, Retrieval-Augmented Generation (RAG) integrates a document retrieval system with an answer generation system. RAG incorporates an initial step where Large Language Models (LLMs) search an external data source to retrieve relevant information before producing text or answering questions [1]. By first retrieving relevant and factual information from external sources and then using that context to generate responses, RAG significantly improves the accuracy of answers and reduces the occurrence of hallucinations. This project demonstrates how combining retrieval with generation leads to more reliable and fact-based question answering.

## 2. Problem Statement

LLMs often generate incorrect facts, provide responses that are not supported by available knowledge sources, and lack transparency in their reasoning process. These limitations can reduce their reliability in critical applications such as education and information retrieval. The primary objective of this system is to address these challenges by developing a framework that retrieves relevant documents prior to generating an answer, thereby grounding responses in verified information. Additionally, the system aims to detect potential hallucinations and compute confidence scores for each generated response, improving both the accuracy and trustworthiness of the outputs in a professional and reliable manner. Compares QA with and without retrieval

.

## 3. Methodology

The methodology of this system is structured into five key steps. First, in the data preparation phase, a dataset comprising simple factual sentences is created, including information such as the location of the Eiffel Tower, capital cities, and other basic factual statements. In the second step, text embedding is performed using the Sentence Transformer model (all-MiniLM-L6-v2), which converts the textual data into high-dimensional vector representations suitable for semantic comparison.

In the third step, a vector database is constructed using FAISS, enabling efficient storage and fast retrieval of document embeddings based on similarity search. The fourth step involves the retrieval process, where each user query is converted into a vector representation, and the system identifies the most semantically similar document chunk by performing a nearest-neighbor search, returning the top-1 matching result.

Finally, in the answer generation stage, the system operates in two modes: in the basic question-answering setup, predefined answers are returned directly, whereas in the RAG approach, the system generates responses based on the retrieved document, ensuring that the output is grounded in relevant and factual information. The Fig. 1representing the methodology is shown below.

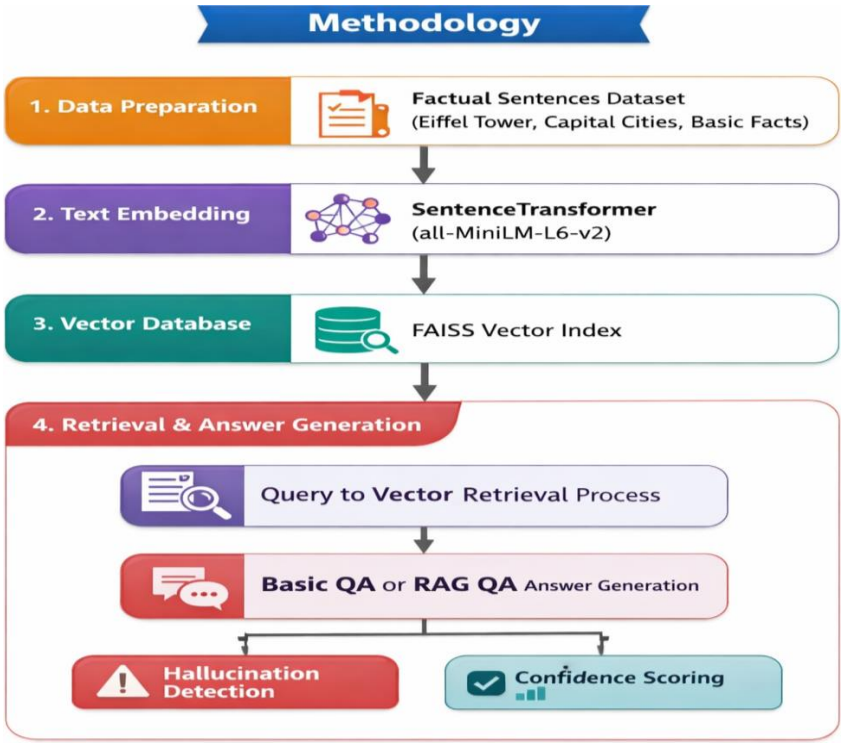


**Fig. 1** Flowchart of the Retrieval-Augmented Generation (RAG) Based Question Answering System

## 4. System Architecture

The system is composed of four main components that work together to ensure accurate and reliable question answering. The first component is the document store, which serves as a small knowledge base containing factual information stored in a file named *documents.txt*. The second component is the retrieval module, which utilizes a FAISS vector database along with sentence embeddings to efficiently retrieve the most relevant information based on semantic similarity. The third component is the question-answering system, which operates in two modes: a basic rule-based QA system that provides predefined responses, and a RAG QA system that generates answers using retrieved contextual information. The fourth component is the evaluation module, which is responsible for assessing the system's performance by detecting potential hallucinations and computing confidence scores to measure the reliability of the generated responses

## 5. Implementation Tools

The system is implemented using Python as the primary programming language due to its simplicity and strong support for AI development. LangChain is used to build and manage the workflow of language model applications, enabling smooth integration between retrieval and generation components. FAISS serves as the vector database for efficient similarity search over text embeddings, while Sentence Transformers are used to convert text into dense vector representations for semantic matching. Visual Studio Code (VS Code) is used as the development environment, providing a convenient platform for coding, testing, and debugging the project.

## 6. Algorithms Used

The system uses cosine similarity for retrieval to measure the semantic closeness between the query and stored document embeddings, enabling effective matching of relevant information. Word overlap is used for confidence scoring by evaluating the similarity between generated answers and retrieved text based on shared terms. Additionally, a rule-based hallucination detection algorithm is applied to identify potential inaccuracies by checking whether the generated response is sufficiently supported by the retrieved content, ensuring improved factual reliability.

## 7. Results

The example output demonstrates the comparison between a basic question-answering system and a RAG approach. For the same question, both systems provide correct

responses; however, the RAG-based system produces a more grounded and contextually reliable answer by relying on retrieved knowledge. The hallucination detection module confirms that no hallucination is present in the response, and the confidence scoring indicates a high level of certainty (0.92–1.0), showing that the answer is strongly supported by the underlying data. The Table 1 below shows the output

**Table 1:** This section presents the components and the resulting outputs

| Components | Output |
|---|---|
| Question | Where is the Eiffel Tower? |
| Basic QA | The Eiffel Tower is in Paris. |
| RAG QA | The Eiffel Tower is located in Paris. |
| Hallucination Detection | False (No hallucination) |
| Confidence Score | 0.92 – 1.0 (High confidence) |

## 8. Comparison Table

The comparison highlights the key differences between a basic question-answering system and a RAG system. While the basic QA model directly generates responses without considering external context, the RAG approach enhances performance by retrieving relevant information before generating answers. This leads to improved accuracy, better contextual understanding, and significantly reduced hallucination risk. Additionally, the RAG system offers higher explainability since its responses are grounded in retrieved documents, making the reasoning process more transparent compared to traditional QA methods. The Table 2 below shows comparison of results.

**Table 2 :** Comparative Analysis of Basic QA and RAG QA in Terms of Key Performance Features

| Feature | Basic QA | RAG QA |
|---|---|---|
| Context Awareness | No | Yes |
| Accuracy | Medium | High |
| Hallucination Risk | High | Low |
| Explainability | Low | High |

## 9. Conclusion

This project successfully demonstrates that RAG significantly reduces hallucinations in question-answering systems. By grounding responses in a structured and reliable knowledge base through a retrieval mechanism, the system ensures that generated answers are factually supported. As a result, it achieves higher accuracy, improved reliability, and better overall performance compared to traditional basic question-answering approaches.

## References

1. Arslan, M., et al., *A Survey on RAG with LLMs.* Procedia computer science, 2024. **246**: p. 3781-3790.